

Starter Code Notes

main.py

Main script for the program

SIM_SERVER_UDP_IP, SIM_SERVER_UDP_PORT

- Simulator UDP endpoint for MAVLink. Gets sent into setup_components() to establish the connection.

system_boot_ms

- Local boot timestamp in ms when the script starts running

shared_data

- Shared runtime dictionary for components. Sent in setup_components(). Unused in starter code

components

- Dict returned by setup_components() with the controller, ts_loop, mavlink_rx, and vision_rx objects

controller, ts_loop, mavlink_rx, vision_rx

- Component object references used by the main loop

is_running

- Main loop flag

Behavior:

- Starts the controller, controller.arm() the drone, enters controller.update() loop
- Joins component threads on exit for safe cleanup

How does this need to be changed since there's a countdown?

setup.py

Initializes components for the main program

setup_components(shared_data, system_boot_ms, server_ip, server_udp_port)

- Initializes components for the main program
- Opens MAVLink UDP connection to simulator
- Creates MAVLink receiver (rx), timesync loop, vision receiver, controller

controller.py

Used for controlling the drone

MAVLINK_CMD_SIM_RESET

- Custom MAVLink reset command ID

MOTOR_FRONT_LEFT, MOTOR_FRONT_RIGHT, MOTOR_BACK_LEFT,
MOTOR_BACK_RIGHT

- Motor output values for actuator control

PITCH_RATE, ROLL_RATE, YAW_RATE, THRUST

- Fixed attitude/rate control targets

RATES_ATTITUDE_MASK

- MAVLink attitude ignore mask

VELOCITY_POSITION_MASK

- MAVLink position target ignore mask

CONTROL_HZ

- Control loop frequency

update_motor_control(mavlink_conn, system_boot_ms)

- Sends actuator control target with motor RPM values

update_attitude_flight_control(mavlink_conn, system_boot_ms)

- Sends desired attitude target message

update_position_flight_control(mavlink_conn, system_boot_ms)

- Sends local NED velocity position target message

Controller

- __init__(sim_conn, data, system_boot_ms)
- update()
 - Calls `update\_motor\_control` and sleeps to maintain control rate
- arm()
 - Sends arm command to the vehicle
- send_sim_reset_command()
 - Sends simulator reset command

What is attitude? What is the purpose of the ignore mask? What is the actuator control target?

mavlink_rx.py

Used for receiving mavlink packets via UDP connection

ENCAPSULATED_RACE_STATUS_MSG_ID, ENCAPSULATED_TRACK_INFO_MSG_ID

- Payload type identifiers for custom encapsulated data

MAVLinkRX

- __init__(mavlink_connection, data)
- create_mavlink_rx(cls, mavlink_connection, data)
 - Starts MAVLink receive thread
- get_thread_for_join()
 - Stops receiver loop and returns thread for join
- mavlink_receive_loop()
 - Receives MAVLink messages and dispatches by type
- on_heartbeat(msg)
 - Parses heartbeat armed state

- `on_timesync(msg)`
 - Parses timesync payload fields
- `on_attitude(msg)`
 - Parses attitude Euler and rates
- `on_local_position_ned(msg)`
 - Parses local position and velocity
- `on_odometry(msg)`
 - Parses odometry pose, velocity, and reset count
- `on_highres_imu(msg)`
 - Parses IMU acceleration and gyro values
- `on_encapsulated_data(msg)`
 - Routes custom encapsulated messages
- `on_race_status(msg)`
 - Unpacks race status packet
- `on_track_data_packet(msg)`
 - Reassembles chunked track data packets
- `on_track_data(payload)`
 - Parses complete track gate payload
- `on_actuator_output_status(msg)`
 - Parses actuator motor outputs
- `on_collision(msg)`
 - Parses collision event and threat data

vision_rx.py

Used for receiving vision UDP dataframes in JPEG for processing

`SIM_SERVER_UDP_IP`, `SIM_SERVER_UDP_PORT`

- UDP endpoint for receiving vision frame packets

VisionRX

- `__init__(data)`
 - Starts UDP receive thread
- `get_thread_for_join()`
 - Stops vision loop and returns thread
- `_vision_loop()`
 - Receives UDP packets, reassembles JPEG frames, decodes images
- `process_frame(frame_id, img)`
 - Placeholder for FPV image processing

timesync.py

Not sure

`TIMESYNC_REQUEST_HZ`

- Frequency for timesync requests

TimeSync

- `__init__(mavlink_connection, data)`
- `create_timesync(cls, mavlink_connection, data)`
 - Starts timesync thread
- `get_thread_for_join()`
 - Stops timesync loop and returns thread
- `timesync_loop()`
 - Periodically sends MAVLink TIMESYNC requests

Questions and Answers

How does main need to be changed since there's a countdown?

What is attitude?

Attitude is important because it controls how thrust direction changes, changing motion. Attitude != Position, it describes orientation.



- Roll is x, Pitch is y, Yaw is z
- Roll is tilt left and right
- Pitch is tilt forward and back
- Yaw is rotation around the vertical axis (heading/direction nose is pointing) like spinning a top

Drones typically estimate angles with sensors. Gyroscope gives angular velocity (speed of rotation + direction), Accelerometer gives gravity direction (gives tilt info), and sometimes a Magnetometer which gives absolute heading (yaw reference).

What is the purpose of the ignore mask?

Essentially, telling the `set_attitude_target_send()` function to accept rates of rotation and thrust. Also telling the `set_position_target_ned_send()` function to accept velocity values.

`RATES_ATTITUDE_MASK` and `VELOCITY_POSITION_MASK` are bitmasks that tell the MAVLink flight controller which fields in the messages to ignore.

The `RATES_ATTITUDE_MASK` in `set_attitude_target_send()` tells the function to ignore the quaternion attitude field and only use the body rate fields + thrust. This makes the message a rate command rather than an explicit attitude being set.

The `VELOCITY_POSITION_MASK` is set to ignore position, acceleration, yaw and yaw rate. Only velocity fields are used in `set_position_target_ned_send()` so we're commanding a velocity rather than a position.

Masks let the controller send only values it wants the vehicle to act on. This avoids conflicts with leftover default values. In theory, this changes the drone behavior accordingly in the `controller.update()` function. Velocity might be the easiest strategy for this.

What is the actuator control target?

Starter code only updates motor controls (actuator target) which isn't super useful. Position flight control allows velocity targets. Basically the way mavlink can speak to the simulator to tell the drone how to behave.

What is the purpose of TimeSync?

- Synchronizes the host clock with the MAVLink simulator
- [setup.py](#) creates the timesync component in setup
- `Mavlink_rx.py` listens for TIMESYNC messages and routes to `on_timesync`. Code does not currently compute the clock offset so it's partially implemented.
- Copilot responded saying it is important for consistent logging, sensor fusion, control loop timing, and correlating events
 - Aligned timestamps allow comparing data from different sources and replaying events accurately
 - Fused sensor data (combined readings into a single estimate) must be time matched otherwise measurements don't correspond to the same real-world instant
 - Controllers rely on precise time intervals for correct update rates and behavior stability
 - Synchronized clocks let you link actions, sensor events, and MAVLink messages reliably

How does it work?

[timesync.py](#) sends MAVLink TIMESYNC messages periodically at 10Hz. It gets the current timestamp with `time.time_ns()`, sends a stamp as `tc1` and `ts1 = 0` to indicate a message is a request. The remote side responds with its own timestamp to compute clock offset and latency.

Manual Control Notes

Attempting to use the set velocity targets to implement up and down with WASD movement.